

Windows Service Development

Option 2 – "Paranoid Canary File" Tamper Detector

Opgave & Emne	Obligatorisk Opgave 2 – Windows Service i PowerShell
Valgt Option	Option 2: "Paranoid Canary File" Tamper Detector
Studerende	Jon Theis Nilsson & Jørgen Helmer Dyhr
Programmeringssprog	PowerShell
Afleveringsdato	Torsdag den 8. oktober 2026

1. Indledning og Systembeskrivelse

Denne rapport dokumenterer udviklingen af en Windows-baggrundsservice i PowerShell med udgangspunkt i Option 2: "Paranoid Canary File" Tamper Detector.

Hensigten er en service, som kontinuerligt overvåger en udvalgt "canary file" mod uautoriseret ændring eller sletning. Ved registrerede afvigelser udføres en automatisk *silent restore* ud fra en sikkerhedskopi, og hændelsen logges kronologisk med tidsstempel.

Rapporten gennemgår servicens arkitektur, logningsstruktur samt test og verifikation af systemet.

2. Arkitektur & PowerShell Service Opbygning

Scriptet er bygget op som et simpelt overvågningsscript der overordnet håndteres i en Windows Service, som tillader at scriptet kan køre i en baggrundstråd.

Efter opstart af scriptet afvikles et while-loop. Så længe servicen er aktiv, kontrolleres målfilen hvert 5. sekund. Brudt ned i trin:

1. Hashværdien af backup-filen beregnes og tilskrives en variabel.
2. While-loop startes.
3. Registrering af tidspunkt under brug af "Get-Date", som anvendes ved logning.
4. Scriptet undersøger om filen eksisterer ved anvendelse af "Test-Path".
5. Hvis filen eksisterer, beregnes hashværdi med "Get-FileHash" af målfilen.

6. Målfilens aktuelle hashværdi sammenholdes med hashværdien af backup-filen. Registrering i loggen.
7. Hvis hashværdien afviger registreres den ændrede hashværdi i loggen.
8. Ved ændring i hashværdi kopieres backup-filen tilbage til målfilens placering -> Der anvendes "Force" for at minimere risikoen for fejl, da det tillader kopiering af read-only filer.
9. Hvis "Test-Path" viser at filen ikke eksisterer genskabes den på baggrund af backup-filen. Dette registreres i loggen.
10. Der itereres over en folder "extern", hvor der beregnes hashværdier af samtlige filer, som så sammenlignes med hashværdien af backup-filen. (Dette skal simulere et eksternt drev).
11. Hvis målfilen er kopieret til "extern" logges dette.
12. Efter kontrollen venter scriptet i 5 sekunder.

3. Logning og Fejlhåndtering

Vi har i forbindelse med logning valgt at bruge cmdlet "Add-Content", da det er en simpel måde at skrive til en fil (og ikke til konsollen), ligesom det fungerer godt i en baggrundstjeneste. Desuden er udgangspunktet, at "Add-Content" append'er og ikke overskriver.

I forhold til genopretning ved evt. crash, så fremgår det af linje 39 i install-filen, at Windows skal forsøge genstart af servicen af 3 omgange med 5 sekunders interval.

```
10/07/2026 19:57:35 Filen findes og hash matcher
10/07/2026 19:57:35 Filen kopieret til eksternt drev
10/07/2026 19:57:41 Filen findes og hash matcher
10/07/2026 19:57:41 Filen kopieret til eksternt drev
10/07/2026 19:57:46 Filen findes og hash matcher
10/07/2026 19:57:46 Filen kopieret til eksternt drev
10/07/2026 19:57:51 Filen findes og hash matcher
10/07/2026 19:57:56 Filen findes og hash matcher
10/07/2026 19:58:01 Filen findes og hash matcher
```

4. Tanker & Forbedringer

Hvor vi i scriptet alene overvåger filerne i en specifik mappe for hashværdier, som matcher backupfilen, så skal det ses som et "proof-of-concept".

I en mere færdig udgave af scriptet har vi kigget ind i brugen af Windows Management Instrumentation (WMI), som jf. Google skulle tillade, at man kan filtrere på events, som udløses når en USB-enhed forbindes til systemet. Dette ville give os en "trigger" i forhold til at målrette et kontinuerligt scan på eksterne enheder og sammenholde hashværdier af indholdet med vores målfil.

Brugen af "event triggers" kunne formentlig også anvendes til selve hovedscriptet, da Windows .NET platform understøtter en klasse i form af System.IO.FileSystemWatcher, som så kunne sættes til at reagere på f.eks. "Changed" og "Deleted".

Hvis det så konstateres, at filen er kopieret til et eksternt drev kunne man overveje, at lade scriptet læse filen og erstatte indholdet med tilfældige data i samme størrelsesorden, som den oprindelige fil – dette vil minimere, men ikke nødvendigvis umuliggøre, risikoen for en forensic recovery.

For at lave relative paths har vi brugt `$MyInvocation.MyCommand.Path` som returnerer path til det script der bliver kørt. Vi troede først at installationen af en service pegede på det sted hvor scriptet lå oprindeligt, men det kopierer hele scriptet, som så bliver kørt fra installations path'en, så `$MyInvocation.MyCommand.Path` er overflødig, og vi kunne blot have brugt `$installFolder` i stedet. Vi har beholdt den som eksemplet, da den virker god at kunne.

I stedet for at laves vores egen log kunne vi kigge på Windows indbyggede Windows Event Log og Write-EventLog. Måske det vil være en bedre løsning.

Vi har prøvet at køre en Task, når filen er blevet kopieret som en slags alert, men det blev lidt "spammy", så den er udkommenteret, men det virker.

Det ville måske give mere mening at køre hele scriptet som en scheduled task i stedet for en service, når vi alligevel køre det i et fastlagt interval. Vi bruger sleep og en endless-loop, som vi er klar over, er en lidt naiv tilgang til problemet, men også fint nok til at forstå essensen af hvordan man bruger services.